bundle command group

Learn how to use the Databricks CLI to deploy and to run jobs and Lakeflow Spark Declarative Pipelines.

14 min. read · View original

The `bundle` command group within the Databricks CLI contains commands for managing Databricks Asset Bundles. Databricks Asset Bundles let you express projects as code and programmatically validate, deploy, and run Databricks workflows such as Databricks jobs, Lakeflow Spark Declarative Pipelines, and MLOps Stacks. See What are Databricks Asset Bundles?.

note

Bundle commands use the settings in `databricks.yml` for authentication when run from inside the bundle folder. If you want to run bundle commands with different authentication from inside the bundle folder, specify a configuration profile using the `--profile` (or `-p`) flag and do not specify a `--target`.

Alternatively, run commands that do not need the same authentication as the bundle from outside the bundle folder.

databricks bundle deploy

Deploy a bundle to the remote workspace.

Bundle target and identity

To deploy the bundle to a specific target, set the `-t` (or `--target`) option along with the target's name as declared within the bundle configuration files. If no command options are specified, the default target as declared within the bundle configuration files is used. For example, for a target declared with the name `dev`:

A bundle can be deployed to multiple workspaces, such as development, staging, and production workspaces. Fundamentally, the `root_path` property is what determines a bundle's unique identity, which defaults to `~/.bundle/${bundle.name}/${bundle.target}`. Therefore by default, a bundle's identity is comprised of the identity of the deployer, the bundle's name, and the bundle's target name. If these are identical across different bundles, deployment of these bundles will interfere with one another.

Furthermore, a bundle deployment tracks the resources it creates in the target workspace by their IDs as a state that is stored in the workspace file system. Resource names are not used to correlate between a bundle deployment and a resource instance, so:

- If a resource in the bundle configuration does not exist in the target workspace, it is created.
- If a resource in the bundle configuration exists in the target workspace, it is updated in the workspace.
- If a resource is removed from the bundle configuration, it is removed from the target workspace if it was previously deployed.
- A resource's association with a bundle can only be forgotten if you change the bundle name, the bundle target, or the workspace. You can run `bundle validate` to output a summary containing these values.

Options

`--auto-approve`

Skip interactive approvals that might be required for deployment.

`-c, --cluster-id string`

Override cluster in the deployment with the given cluster ID.

`--fail-on-active-runs`

Fail if there are running jobs or pipelines in the deployment.

`--force`

Force-override Git branch validation.

`--force-lock`

Force acquisition of deployment lock. This option disables the mechanism that prevents concurrent deployments from interacting with each other. It should only be used if the previous deployment crashed or was interrupted and left a stale lock file.

`--plan`

Path to a JSON plan file to apply instead of planning (direct engine only). The plan file can be created using `databricks bundle plan -o json`.

Global flags

Examples

The following example deploys a bundle using a specific cluster ID:

databricks bundle deployment

Deployment-related commands.

Available Commands

- `bind` - Bind a bundle-defined resource to an existing resource in the remote workspace.
- `migrate` - Migrate a bundle to use the direct deployment engine.
- `unbind` - Unbind a bundle-defined resource from its remote resource.

databricks bundle deployment bind

Link bundle-defined resources to existing resources in the Databricks workspace so that they become managed by Databricks Asset Bundles. If you bind a resource, the existing Databricks resource in the workspace is updated based on the configuration defined in the bundle it is bound to after the next `bundle deploy`.

Bind does not recreate data. For example, if a pipeline with data in a catalog had bind applied, you can deploy to that pipeline without losing the existing data. In addition, you do not need to recompute the Materialized view, for example, so pipelines do not have to rerun.

The bind command should be used with the `--target` flag. For example, bind your production deployment to your production pipeline using `databricks bundle deployment bind --target prod my_pipeline 7668611149d5709ac9-2906-1229-9956-586a9zed8929`

tip

It's a good idea to confirm the resource in the workspace before running bind.

Bind is supported for the following resources:

For resources supported by the `bundle generate` command, automatically bind the resource after generation using the `--bind` option.

Arguments

KEY

The key of the resource to bind

RESOURCE_ID

The ID of the existing resource to bind to

Options

--auto-approve

Automatically approve the binding, instead of prompting

--force-lock

Force acquisition of deployment lock. This option disables the mechanism that prevents concurrent deployments from interacting with each other. It should only be used if the previous deployment crashed or was interrupted and left a stale lock file.

Global flags

Examples

The following command binds the resource `hello_job` to its remote counterpart in the workspace. The command outputs a diff and allows you to deny the resource binding, but if confirmed, any updates to the job definition in the bundle are applied to the corresponding remote job when the bundle is next deployed.

databricks bundle deployment migrate

Migrate the bundle from using the Terraform deployment engine to using the direct deployment engine. See Migrate to the direct deployment engine. To complete the migration you must then deploy the bundle.

You can verify a migration was successful by running `databricks bundle plan`. See databricks bundle plan.

Arguments

None

Options

Global flags

Examples

The following example migrates the current bundle to use the direct deployment engine:

databricks bundle deployment unbind

Remove the link between the resource in a bundle and its remote counterpart in a workspace.

Arguments

KEY

The key of the resource to unbind

Options

`--force-lock`

Force acquisition of deployment lock. This option disables the mechanism that prevents concurrent deployments from interacting with each other. It should only be used if the previous deployment crashed or was interrupted and left a stale lock file.

Global flags

Examples

The following example unbinds the `hello_job` resource:

databricks bundle destroy

warning

Destroying a bundle permanently deletes a bundle's previously-deployed jobs, pipelines, and artifacts. This action cannot be undone.

Delete jobs, pipelines, other resources, and artifacts that were previously deployed.

note

A bundle's identity is comprised of the bundle name, the bundle target, and the workspace. If you have changed any of these and then attempt to destroy a bundle prior to deploying, an error will occur.

By default, you are prompted to confirm permanent deletion of the previously-deployed jobs, pipelines, and artifacts. To skip these prompts and perform automatic permanent deletion, add the `--auto-approve` option to the `bundle destroy` command.

You can use the `lifecycle` setting to prevent specific resources from being destroyed. See lifecycle.

Options

`--auto-approve`

Skip interactive approvals for deleting resources and files

`--force-lock`

Force acquisition of deployment lock. This option disables the mechanism that prevents concurrent deployments from interacting with each other. It should only be used if the previous deployment crashed or was interrupted and left a stale lock file.

Global flags

Examples

The following command deletes all previously-deployed resources and artifacts that are defined in the bundle configuration files:

databricks bundle generate

Generate bundle configuration for a resource that already exists in your Databricks workspace. The following resources are supported: app, dashboard, job, pipeline.

By default, this command generates a `*.yml` file for the resource in the `resources` folder of the bundle project and also downloads any files, such as notebooks, referenced in the configuration.

important

The `bundle generate` command is provided as a convenience to autogenerate resource configuration. However, if your bundle includes resource configuration and you deploy it, Databricks creates a new resource rather than updating the existing one. To update an existing resource instead, you must either use the `--bind` flag with `bundle generate` or run `bundle deployment bind` before deploying. See databricks bundle deployment bind.

Available Commands

- `app` - Generate bundle configuration for a Databricks app.
- `dashboard` - Generate configuration for a dashboard.
- `job` - Generate bundle configuration for a job.
- `pipeline` - Generate bundle configuration for a pipeline.

Options

`--key string`

Resource key to use for the generated configuration

Global flags

databricks bundle generate app

Generate bundle configuration for an existing Databricks app in the workspace.

Options

`--bind`

Automatically bind the generated resource with the existing one in the workspace.

`-d, --config-dir string`

Directory path where the output bundle config will be stored (default "resources")

`--existing-app-name string`

App name to generate config for

`-f, --force`

Force overwrite existing files in the output directory

`-s, --source-dir string`

Directory path where the app files will be stored (default "src/app")

Global flags

Examples

The following example generates configuration for an existing app named `my-app`. You can get the app name from the **Compute** > **Apps** tab of the workspace UI.

The following command generates a new `hello_world.app.yml` file in the `resources` bundle project folder, and downloads the app's code files, such as the app's command configuration file `app.yaml` and main `app.py`. By default, the code files are copied to the bundle's `src` folder.

YAML

```
# This is the contents of the resulting /resources/hello-world.app.yml file.
resources:
apps:
hello_world:
name: hello-world
description: A basic starter application.
source_code_path: ../src/app
```

databricks bundle generate dashboard

Generate configuration for an existing dashboard in the workspace.

tip

To update the `.lvdash.json` file after you have already deployed a dashboard, use the `--resource` option when you run `bundle generate dashboard` to generate that file for the existing dashboard resource. To continuously poll and retrieve updates to a dashboard, use the `--force` and `--watch` options.

Options

`--bind`

Automatically bind the generated resource with the existing one in the workspace.

`-s, --dashboard-dir string`

Directory to write the dashboard representation to (default "src")

`--existing-id string`

ID of the dashboard to generate configuration for

`--existing-path string`

Workspace path of the dashboard to generate configuration for

`-f, --force`

Force overwrite existing files in the output directory

`--resource string`

Resource key of dashboard to watch for changes

`-d, --resource-dir string`

Directory to write the configuration to (default "resources")

`--watch`

Watch for changes to the dashboard and update the configuration

Global flags

Examples

The following example generates configuration by an existing dashboard ID:

You can also generate configuration for an existing dashboard by workspace path. Copy the workspace path for a dashboard from the workspace UI.

For example, the following command generates a new `baby_gender_by_county.dashboard.yml` file in the `resources` bundle project folder containing the YAML below, and downloads the `baby_gender_by_county.lvdash.json` file to the `src` project folder.

Bash

```
databricks bundle generate dashboard --existing-path
"/Workspace/Users/someone@example.com/baby_gender_by_county.lvdash.json"
```

YAML

```
# This is the contents of the resulting baby_gender_by_county.dashboard.yml file.
resources:
dashboards:
baby_gender_by_county:
display_name:'Baby gender by county'
warehouse_id: aae11o8e6fe9zz79
file_path: ../src/baby_gender_by_county.lvdash.json
```

databricks bundle generate job

Generate bundle configuration for a job.

note

Currently, only jobs with notebook tasks are supported by this command.

Options

`--bind`

Automatically bind the generated resource with the existing one in the workspace.

`-d, --config-dir string`

Dir path where the output config will be stored (default "resources")

`--existing-job-id int`

Job ID of the job to generate config for

`-f, --force`

Force overwrite existing files in the output directory

`-s, --source-dir string`

Dir path where the downloaded files will be stored (default "src")

Global flags

Examples

The following example generates a new `hello_job.yml` file in the `resources` bundle project folder containing the YAML below, and downloads the `simple_notebook.py` to the `src` project folder. It also binds the generated resource with the existing job in the workspace.

YAML

```
# This is the contents of the resulting hello_job.yml file.
resources:
jobs:
hello_job:
name:'Hello Job'
```

```
tasks:
-task_key: run_notebook
email_notifications:{}
notebook_task:
notebook_path: ../src/simple_notebook.py
source: WORKSPACE
run_if: ALL_SUCCESS
max_concurrent_runs:1
```

databricks bundle generate pipeline

Generate bundle configuration for an existing pipeline.

tip

If you have an existing Spark Declarative Pipelines (SDP) project, you can generate configuration for it using `databricks pipelines generate`. See databricks pipelines generate.

Options

`--bind`

Automatically bind the generated resource with the existing one in the workspace.

`-d, --config-dir string`

Dir path where the output config will be stored (default "resources")

`--existing-pipeline-id string`

ID of the pipeline to generate config for

`-f, --force`

Force overwrite existing files in the output directory

`-s, --source-dir string`

Dir path where the downloaded files will be stored (default "src")

Global flags

Examples

The following example generates configuration for an existing pipeline:

databricks bundle init

Initialize a new bundle using a bundle template. Templates can be configured to prompt the user for values. See Databricks Asset Bundle project templates.

Arguments

`TEMPLATE_PATH`

Template to use for initialization (optional)

Options

`--branch string`

Git branch to use for template initialization

`--config-file string`

JSON file containing key value pairs of input parameters required for template initialization.

`--output-dir string`

Directory to write the initialized template to.

`--tag string`

Git tag to use for template initialization

`--template-dir string`

Directory path within a Git repository containing the template.

Global flags

Examples

The following example prompts with a list of default bundle templates from which to choose:

The following example initializes a bundle using the default Python template:

To create a Databricks Asset Bundle using a custom Databricks Asset Bundle template, specify the custom template path:

Bash

```
databricks bundle init <project-template-local-path-or-url>\
--project-dir="</local/path/to/project/template/output>"
```

The following example initializes a bundle from a Git repository:

The following example initializes with a specific branch:

databricks bundle open

Navigate to a bundle resource in the workspace, specifying the resource to open. If a resource key is not specified, this command outputs a list of the bundle's resources from which to choose.

Options

`--force-pull`

Skip local cache and load the state from the remote workspace

Global flags

Examples

The following example launches a browser and navigates to the `baby_gender_by_county` dashboard in the bundle in the Databricks workspace that is configured for the bundle:

databricks bundle plan

Show the deployment plan for the current bundle configuration.

This command builds the bundle and displays the actions which will be performed on resources that would be deployed, without making any changes. This allows you to preview changes before running `bundle deploy`.

Options

`-c, --cluster-id string`

Override cluster in the deployment with the given cluster ID.

`--force`

Force-override Git branch validation.

Global flags

Examples

The following example outputs the deployment plan for a bundle that builds a Python wheel, and defines a job and a pipeline:

Output

```
Building python_artifact...
create jobs.my_bundle_job
create pipelines.my_bundle_pipeline
```

databricks bundle run

Run a job, pipeline, or script. If you don't specify a resource, the command prompts with defined jobs, pipelines, and scripts from which to choose. Alternatively, specify the job or pipeline key or script name declared within the bundle configuration files.

Validate a pipeline

If you want to do a pipeline validation run, use the `--validate-only` option, as shown in the following example:

Pass job parameters

To pass job parameters, use the `--params` option, followed by comma-separated key-value pairs, where the key is the parameter name. For example, the following command sets the parameter with the name `message` to `HelloWorld` for the job `hello_job`:

note

As shown in the following examples, you can pass parameters to job tasks using the job task options, but the `--params` option is the recommended method for passing job parameters. An error occurs if job parameters are specified for a job that doesn't have job parameters defined or if task parameters are specified for a job that has job parameters defined.

You can also specify keyword or positional arguments. If the specified job uses job parameters or the job has a notebook task with parameters, flag names are mapped to the parameter names:

Or if the specified job does not use job parameters and the job has a Python file task or a Python wheel task:

For an example job definition with parameters, see Job with parameters.

Execute scripts

To execute scripts such as integration tests with a bundle's configured authentication credentials, you can either run scripts inline or run a script defined in the bundle configuration. Scripts are run using the same authentication context configured in the bundle.

- Append a double hyphen (`--`) after `bundle run` to run scripts inline. For example, the following command outputs the current user's current working directory:

- Alternatively, define a script within the `scripts` mapping in your bundle configuration, then use `bundle run` to run the script:

  For more information about `scripts` configuration, see scripts.

Bundle authentication information is passed to child processes using environment variables. See Databricks unified authentication.

Arguments

KEY

The unique identifier of the resource to run (optional)

Options

`--no-wait`

Don't wait for the run to complete.

`--restart`

Restart the run if it is already running.

Global flags

Job Flags

The following flags are job-level parameter flags. See Configure job parameters.

`--params stringToString`

comma separated k=v pairs for job parameters (default `[]`)

Job Task Flags

The following flags are task-level parameter flags. See Configure task parameters. Databricks recommends using job-level parameters (`--params`) over task-level parameters.

`--dbt-commands strings`

A list of commands to execute for jobs with DBT tasks.

`--jar-params strings`

A list of parameters for jobs with Spark JAR tasks.

`--notebook-params stringToString`

A map from keys to values for jobs with notebook tasks. (default `[]`)

`--pipeline-params stringToString`

A map from keys to values for jobs with pipeline tasks. (default `[]`)

`--python-named-params stringToString`

A map from keys to values for jobs with Python wheel tasks. (default `[]`)

`--python-params strings`

A list of parameters for jobs with Python tasks.

`--spark-submit-params strings`

A list of parameters for jobs with Spark submit tasks.

`--sql-params stringToString`

A map from keys to values for jobs with SQL tasks. (default `[]`)

Pipeline Flags

The following flags are pipeline flags.

`--full-refresh strings`

List of tables to reset and recompute.

`--full-refresh-all`

Perform a full graph reset and recompute.

`--refresh strings`

List of tables to update.

`--refresh-all`

Perform a full graph update.

`--validate-only`

Perform an update to validate graph correctness.

Examples

The following example runs a job `hello_job` in the default target:

The following example runs a job `hello_job` within the context of a target declared with the name `dev`:

The following example cancels and restarts an existing job run:

The following example runs a pipeline with full refresh:

The following example executes a command in the bundle context:

databricks bundle schema

Display JSON Schema for the bundle configuration.

Options

Global flags

Examples

The following example outputs the JSON schema for the bundle configuration:

To output the bundle configuration schema as a JSON file, run the `bundle schema` command and redirect the output to a JSON file. For example, you can generate a file named `bundle_config_schema.json` within the current directory:

databricks bundle summary

Output a summary of a bundle's identity and resources, including deep links for resources so that you can easily navigate to the resource in the Databricks workspace.

tip

You can also use `bundle open` to navigate to a resource in the Databricks workspace. See databricks bundle open.

Options

`--force-pull`

Skip local cache and load the state from the remote workspace

Global flags

Examples

The following example outputs a summary of a bundle's deployed resources:

The following output is the summary of a bundle named `my_pipeline_bundle` that defines a job and a pipeline:

```
Name: my_pipeline_bundle
Target: dev
Workspace:
  Host: https://myworkspace.cloud.databricks.com
  User: someone@example.com
  Path: /Users/someone@example.com/.bundle/my_pipeline/dev
Resources:
  Jobs:
    my_project_job:
      Name: [dev someone] my_project_job
      URL:  https://myworkspace.cloud.databricks.com/jobs/206000809187888?o=6051000018419999
  Pipelines:
    my_project_pipeline:
      Name: [dev someone] my_project_pipeline
      URL:  https://myworkspace.cloud.databricks.com/pipelines/7f559fd5-zztz-47fa-aa5c-c6bf034b4f58?
o=6051000018419999
```

databricks bundle sync

Perform a one-way synchronization of a bundle's file changes within a local filesystem directory, to a directory within a remote Databricks workspace.

note

`bundle sync` commands cannot synchronize file changes from a directory within a remote Databricks workspace, back to a directory within a local filesystem.

`databricks bundle sync` commands work in the same way as `databricks sync` commands and are provided as a productivity convenience. For command usage information, see `sync` command.

Options

`--dry-run`

Simulate sync execution without making actual changes

`--full`

Perform full synchronization (default is incremental)

`--interval duration`

File system polling interval (for `--watch`) (default 1s)

`--output type`

Type of the output format

`--watch`

Watch local file system for changes

Global flags

Examples

The following example performs a dry run sync:

The following example watches for changes and syncs automatically:

The following example performs a full synchronization:

databricks bundle validate

Validate bundle configuration files are syntactically correct.

By default this command returns a summary of the bundle identity:

Output

```
Name: MyBundle
Target: dev
Workspace:
  Host: https://my-host.cloud.databricks.com
  User: someone@example.com
  Path: /Users/someone@example.com/.bundle/MyBundle/dev

Validation OK!
```

note

The `bundle validate` command outputs warnings if resource properties are defined in the bundle configuration files that are not found in the corresponding object's schema.

If you only want to output a summary of the bundle's identity and resources, use bundle summary.

Options

Global flags

Examples

The following example validates the bundle configuration:

Global flags

`--debug`

Whether to enable debug logging.

`-h` or `--help`

Display help for the Databricks CLI or the related command group or the related command.

`--log-file` string

A string representing the file to write output logs to. If this flag is not specified then the default is to write output logs to stderr.

`--log-format` format

The log format type, `text` or `json`. The default value is `text`.

`--log-level` string

A string representing the log format level. If not specified then the log format level is disabled.

`-o, --output` type

The command output type, `text` or `json`. The default value is `text`.

`-p, --profile` string

The name of the profile in the `~/.databrickscfg` file to use to run the command. If this flag is not specified then if it exists, the profile named `DEFAULT` is used.

`--progress-format` format

The format to display progress logs: `default`, `append`, `inplace`, or `json`

`-t, --target` string

If applicable, the bundle target to use

`--var strings`

set values for variables defined in bundle config. Example: `--var="foo=bar"`